

00 - Introduction

C# (sharp) - обектно-ориентиран език, създаден и раз-
виван от Microsoft в нова форма на .Net Framework.
C# най-често се използва във Windows и не се разпрости-
рява универсално, а е част от .Net.
Със C# се създават RTH - Rich Internet Applications.

.Net Framework - среда за разработка и изпълнение
на програми, написани на .Net езика като C#, F#...
Платформата включва:

CLR - Common Language Runtime - среда за разпрос-
транение и изпълнение на управляем код
(Managed Code). Това е виртуална машина
от която приложението не може да излезе, освен
чрез разрешение от user-а като Save As... Междимашин
код се нар. IL в Reflector и е средно ниво или асемблер
и език от високо ниво. Чрез него се ползва себеси-
мост или .Net езика. CLR автоматично управ-
лява паметта (заделя, после освобождава чрез Garbage
Collector). Виртуална машина или виртуален процесор.

CSC - компилатор, който превръща C# програмите в
производимия за CLR междинен код MSIL
(Microsoft Intermediate Language).

FCI - Framework Class Library = стандартни библиотеки

+ BCL - Base Class Library - основни библиотеки (core-и)

+ Библиотеки за достъп до данни:

- ADO .Net - достъп до БД (MS SQL или MySQL)...

- LINQ (Language) - Language Integrated Query (изгради въпросителни)

- XML - Extensible Markup Language

Дебюти

.Net (сформировано 2002 г.)

Java

C# | C++ | VB.Net | J# | F# | JScript | Perl | Delphi | ...

ASP.Net

Web Forms, MVC, AJAX Windows Forms WPF Silverlight
Mobile Internet Toolkit

WCF and WF (communication and workflow Peer)

Ado.Net, LINQ and XML (Data Peer)

Base Class Library (BCL)

Common Language Runtime (CLR)

Operating System (OS)

FCL

CLR

WCF и Windows Forms - उपयोगकर्ता को कोड को रनटाइम में एक्सेस करने के लिए उपयोग करता है

ASP.Net - framework - यह एक ऐसा फ्रेमवर्क है जो ASP-Active Server Pages

Web Forms - यह एक वेब फॉर्म है जो उपयोगकर्ता को वेब पर डेटा दर्ज करने के लिए उपयोग करता है

MVC (Model - View - Controller) - Model - डेटा, View - प्रदर्शन, Controller - नियंत्रण

AJAX - (Asynchronous JavaScript and XML) - यह एक तकनीक है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

Mobile Internet Toolkit - यह एक फ्रेमवर्क है जो ASP.Net के लिए मोबाइल डिवाइसों के लिए उपयोग करता है

WPF - Windows Presentation Foundation - यह एक फ्रेमवर्क है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

Silverlight - यह एक फ्रेमवर्क है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

MSDN Library - यह एक वेबसाइट है जो ASP.Net के लिए उपयोगकर्ता को सहायता प्रदान करती है

ORM - Object-Relational Mapping - यह एक तकनीक है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

CMS - Content Management System - यह एक फ्रेमवर्क है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

APM - Agile Project Management - यह एक फ्रेमवर्क है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

reporting - यह एक फ्रेमवर्क है जो वेब पर डेटा को बिना पूरी पृष्ठ को रिफ्रेश किए बिना अपडेट करती है

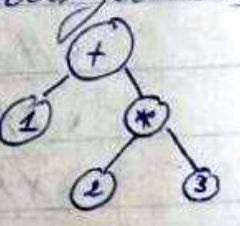
Объекты - массивы, списки и др. - имеют стр. характеристики

Методы - подпрограммы в C# - могут да вызываться друг из друга

Рекурсия - это метод вызывает сам себя

Парсер - программа, или часть от программы,

извлекающая синтаксический анализ (превод на линейную последовательность без форматирования графика. Результат обновлен в дзрво)



$$1+2*3 \rightarrow$$

Java - по-стара и разпространена технология от C#, безплатна, с по-големи възможности, но-бавна поради преминаването си, което би довело до прекъсване на много редове код за да се поправи. C# е разработен без тези недостатъци + по-лесен. JVM (Java Virtual Machine) съответства на CLR при .Net.

⇒ .Net е изработен код и сайт лиценз, официално се разпространява във Windows, като по-малко, Free BSD, Mac OS и др. C# програмите могат да се изпълняват във свободното .Net Framework имплементация Mono, която не се поддържа официално от MS.

Среда - Visual C# 2010 Express (безплатна) Edition за C# 4.0 Ultimate (проф.)

C - хордуверно насочен за задачи на OS-и, драйвери, периферни контролери (embedded devices)

C++ дава бърза връзка між хардуер и софтуер в 3D-игри.

PHP - използва се заради простата структура на кода.

Подходящ е за създаване на малък блог с Word Press, малък сайт с Joomla или Drupal, или дискусияния форум с PHPBB.

OpenOffice - резултатът е сходен с MS Office, но усещането е по-малко.

Лекции:
<http://academy.telarix.com/academy-courses/csharp-programming-fundamentals/resources>

Видео: - II - 1 video

Курса: <http://introprogramming.info/intro-csharp-book/>

Ресурси и гр.: <http://groups.google.com/group/telarixacademy>

Reflector

Just Code

Имената на файловете = името на програмата (класа, метода). Вака думата от името е на английски и започва с главна буква "HelloWorld" и трябва да е описателно. Големият метод се разбива на части, които се пише и тества на части. При даване код да го четеш и отговаряш. Всеки клас е в отделен файл. Единствено проблемите вие са с малка. Константите са само с главни букви; C# е key sensitive. Добавянето на цифри и спец. знаци не е желателно. Смяката на името на класа автоматично сменя името на файла. #3

01 - Intro Programming

Управление на компютъра:

- най-ниско ниво - управлява и комуникира CPU и регистрите му. Асемблерните езиги директно вършат работата.
 - на по-високо ниво - свързано с OS: файлови сис., периферни у-ва, мреж. и, комуникационни протоколи и др.
 - на най-високо ниво в софтуера - приложението (утилитен програм).
- Езиги: C#, Java, C++, PHP, Visual Basic, Python, Ruby, Perl и др...

задават не-сложни инструкции, които карат CPU да върши работата.

Етапи при разработката на софтуер (+ документация на всеки етап)

① Избиране на изискванията за продукта и изготвяне на задание (от бизнес аналитици).

② Планиране и изготвяне на архитектура и дизайн (от мн. опитни софтуерни архитекти - платформи, технологии, дизайн)

• вид на приложението - конзолно, графично

GUI - Graphical User Interface application

клиент - сървър приложение, RPA, web, или peer-to-peer приложение.

• архитектура на програмата - еднослойна, многослойна, SOA

• програмни език, или комбинация от езиги

• технологии - платформа - (.Net, Java EE, LAMP и др...)

- сървър за БД - (Oracle, SQL Server, MySQL)

- user interface - (Flash, VFP, Silverlight, ASP.Net,

Eclipse RCP, Java Server Faces, Windows Forms)

- брзи и уменията на разработчиците;

- план на разработката - етапи, ресурси, срокове;

- други - големина, местоположение и комуникация на екипа

③ Реализация (имплементация) - писане на сурс кода в програмния език

④ Изпитания - чрез тестове с авт. програми. Реализира се от QA - Quality Assurance = инженерите по осигуряване на качеството

⑤ Внедряване (deployment) чрез програма-инсталатор от адм. на БД (DBA)

⑥ Поддръжка - от експерти по поддръжката - актуализ. на данни, състояние на хардуер

0. Компютърно програмиране - създаване на поредица от инструкции, казващи на компютъра какво да прави

Библиотека (using...)

Пространство от имена (namespace) - ^{при class не е необл.} в него се съхраняват пр.

Клас = контейнер за методи (class)

Метод = поредица от команди, които класът трябва да изпълни.

В него се съхраняват конструкции (холомиди).

Дебъгване = тестване, проследяване изн. на програмата

* File → New Project → Console Application → HelloWorld.cs

// Деклариране:

// Библиотека

```
using System;
```

```
namespace HelloWorld
```

// пр-во от имена

```
{
```

```
class HelloWorld
```

// клас

```
{
```

```
static void Main(string[] args) // стартова точка на пр. // метод Main (!)
```

// не е задължително

```
{
```

```
Console.WriteLine("Hello World!"); // конструкция
```

```
}
```

```
}
```

// За * от Java, отваришките и зов варианти са се посивели сами като

Ако библиотека е подготвена → не се използва → може [Del]

Ако не включим библиотека System

⇒ трябва да я извикаме в конструкцията

```
System.Console.WriteLine();
```

* [F2] → преименуване класа

* [F5] = Build + [Ctrl] - за зареждане на конзолата

„От класа Console, намери и извикай метода WriteLine и му кажи да изведе екринът **Hello World!**“

След компилиране на кода от C# се получават
асемблите със същото име, но с разширения .dll и .exe.

* За да видим резултатите:

[Alt] + > Tools → Folder Options → View →

☐ Hide extensions for known file types.

(макар че от метката)

* .exe файла се компира в папката → bin → debug

Може да го копираме и ще работи и на комп. без VS Studio

С които ще се стартира бързо и излезе, но при

[Shift] + J → Open Command Window here →

отваря се CMD с текущата директория.

Пишем първата буква на търсения файл и през [Tab]

кликваме предположителните файлове докато излезе .exe

* Компилира се през [F6], или [Shift + Ctrl + B], или
Build → Build Solution / Project.

- синтактична проверка;

- показан ли е типът на дадените;

превежда кода на MSIL (бинарен код, асембли, през
който се постига съвместимостта на .Net езиките)

* .Net програмите са виртуални, без виртуалната
машина на .Net няма да тръгнат. Създадени са
за виртуалният процесор на .Net, използват
междумашинен език MSIL. Всички програми в .Net
са обектно-ориентирани, които за ≠ от java, 1 програ-
ма може да е съставена от ≠ езика (C# и VBasic пр.)

* Debug → Start Debugging

Breakpoint → [F5] ^{start} → [F10] - ходим по редове...

"[F9] (стопер)

Клик в у грешката в Error List ни отвежда в грешката в кода.

Когато сме на даден ред и неговият код е жалт, значи той
все още не е изпълнен.

След отстраняване на грешките:

* Debug → Stop Debugging, или [Shift + F5]

* Подреждане: [Ctrl + Space + ↑ + Enter]

* По подреждане Help-а не работи [F1]...

Help → Manager Help Settings → Help Library Manager
→ Choose Online/Local Help → ☐ I want to use online help
☒ I want to use local help

* Solution → g. S. → Add → New Project

⇒ добавяме конкретен проект, който са наредили
в 1 Solution ... sln.

За да превърнем наш проект:

Solution → g. S. → Properties → Startup Project →

☒ Current Selection → Ok

☐ Single Startup Project

* За да отворим папката със сурса:

Визуално: c. cs → g. S. → Open Containing Folder

* За да преместим маркиран блок с код:

[Tab] →

[Shift + Tab] ←

* За да се подреди реформатиран код: [Ctrl + E + D]

(или се инсталира Just Code на Telerik → g. S. → Format Code)

* Първи for + [Tab] × 2 ⇒ автомат. изчисляване за for(...)
св + [Tab] × 2 ⇒ Console.WriteLine();

* Автоматично управление на паметта:
 - CLR автоматично зарежда и освобождава памет, което повишава производителността. Не се знае със сигурност в кой момент Garbage Collector изчиства паметта от неизползваните обекти. Изразходването от дадено приложение памет може да се види от Task Monitor → Processes → Memory.
 За да няма зареждане на памет (т.е. програмата да не "гъла" много) в C# управление и ползване на типовете, т.е. на string не може да се присвои число (за * от C и C++).

* File → New Project → WPF → от Toolbox могат да се добавят елементи и с тях да бъдат теглени XAML-и.

* В MSDN-документацията за C# има примери (за Java-и).

* Visual Studio → C# → .Net Framework SDK
 Eclipse → java → JDK

* Др. текстов редактор е emacs → обикн. се пише на Perl
 Dump = Crash in Windows
 Bing = Microsoft Search.

* CMD → g. S → Run as administrator →
 md (създаване директории) "IntroCSharp" = марка
 cd (влизане в нея)

→ notepad HelloCSharp.cs → ! Cannot find ... file.

Do you want to create it? → OK → пишем core → [Ctrl+S]

→ [Alt+F4] → csc HelloCSharp.cs → Ако възникне грешка: копираме .Net им указване пътя до csc:

C:\Windows\Microsoft.NET\Framework\v4.0.2108\csc.exe
→ C:\IntroCSharp\HelloCSharp.exe → готово.

* Проверка на естествена конфигурация в Windows:

Control Panel → System → Advanced System Settings →
→ System Properties → Environment Variables →
→ Path → Edit → посрещане ; и добавяване (copy → Paste)
необходимостта на директория c:\csc.exe) след възникне при
като в CMD командата "csc" ще получи "fatal error"
ако не зададем или на файл за компилиране
(csc HelloWorldApp.cs).

* #. Net езикът могат да обмекнат кода си безпроблемно.
Веднаж написан и компилиран код се изпълнява на
OS и # хордшер. Независимостта от среда е заложена
когато още при създаването. Изходният код се компилира до
CIL - Common Intermediate Language.

Той не се изпълнява директно от микропроцесора,
а от CLR = среда за контролирано изпълнение на
управляем код = "сърцето на .Net.

* Разновидности на .Net платформата

- .Net Framework - най-използван; → всички приложения
Windows програми, уеб приложения и др.
- .Net Compact Framework (CF) - "олекотена" за мобилни
телефони и др PDA-устройства (изр. .Net Mobile Edition)
- Silverlight - "олекотена" - плагин в браузърите за RIA.

* Уеб технологиите (ASP.NET) позволяват писане на динамични
уеб приложения. .Net RIA технологиите (Silverlight) позволяват
писане на богати мултимедийни приложения в Internet среда.

- * Class Libraries (библиотеки от класове) =
 естествени библиотеки със стандартна функционалност:
 - System.Math - мат. ф-ии
 - System.Net - работа с мрежата
 (System.Net.Mail.MailMessage $\Rightarrow$ изпращане e-mail)
 (System.Net.WebClient $\Rightarrow$ изтегляне файлове от Internet)
 - System.Data
 (System.Data.SqlClient $\Rightarrow$ връзка с SQL server чрез ADO.NET)

- * ORM-технологии за обектно-реляционна периментност и
 кои дайки; първоначално са се развивали самостоя-
 телно, но когато набират популярност са вградени в
 .Net 3.5 $\Rightarrow$ LINQ-to-SQL
 .Net 4.0 $\Rightarrow$ ADO.NET Entity

- * В .Net се създават обекти и се викат техниките.

- * API - Application Programming Interface =
 = набор от публични класове и методи, които са
 достъпни за употреба (от програмистите).
 За API за работа с графика, API за работа с принтер, API...

- * Sharp Developer и MonoDevelop са алтернативи на VS Studio.

- * Декомпиляция - процес на генериране на source код
 от F-и извлечени бинарни файл (.Net assembly *.exe; *.dll)
 $\Rightarrow$ Reflector, JustDecompile, IL Spy
 File $\rightarrow$ Open
 $\rightarrow$ Open from GAC (Global Assembly Cache)